Wireshark

Network Analysis Manual

SOC Analyst Track · CyberConnect Fellowship

Ali Waqar Zafar

github.com/aliwaqar04/CyberConnect-Fellowship-SOC-labs

```
Build Wireshark Manual

Reference Wireshark Official Documentation

Tasks:
   1. Purpose of Wireshark
   2. How to Approach a Packet Capture
   3. Protocol Deep Dive. Choose ONE of the following:
              •  DNS
              •  TCP
              •  HTTP
              •  TLS
              •  ICMP
   4. Write a Python script using PyShark that:
              •  Opens a provided .pcap file
              •  Filters for DNS traffic only
      Prints:
              •  Source IP
              •  Queried domain name
```

Figure 1 – Task specification

1. Purpose of Wireshark

Wireshark is the world's leading open-source network protocol analyser. It allows analysts to capture and interactively inspect the traffic running on a computer network in real time, making it an indispensable tool in the SOC toolkit.

Core Capabilities

Live Packet Capture — Wireshark attaches to any network interface and records every frame traversing the wire or airwaves.

Deep Protocol Inspection — Over 3,000 protocols are decoded automatically — from Ethernet and IP all the way up to application-layer protocols like DNS, HTTP and TLS.

Display Filters — Powerful filter expressions let analysts focus on exactly the traffic they care about (e.g. `dns`, `tcp.port==443`).

Offline Analysis — Captured traffic is saved as industry-standard **.pcap** files that can be shared, replayed, or processed by scripts.

Statistics & Visualisation — Built-in tools produce protocol hierarchy trees, I/O graphs, conversation lists, and expert-info summaries.

Forensics & Incident Response — SOC analysts use Wireshark to reconstruct events, hunt for indicators of compromise (IoCs), and verify remediation.

■ **SOC Use-Case:** When an alert fires on the SIEM, a Wireshark capture from the affected host lets you validate whether the alert is a true positive, understand the attack chain, and identify any exfiltrated data — all without relying solely on log files.

Wireshark Home Screen

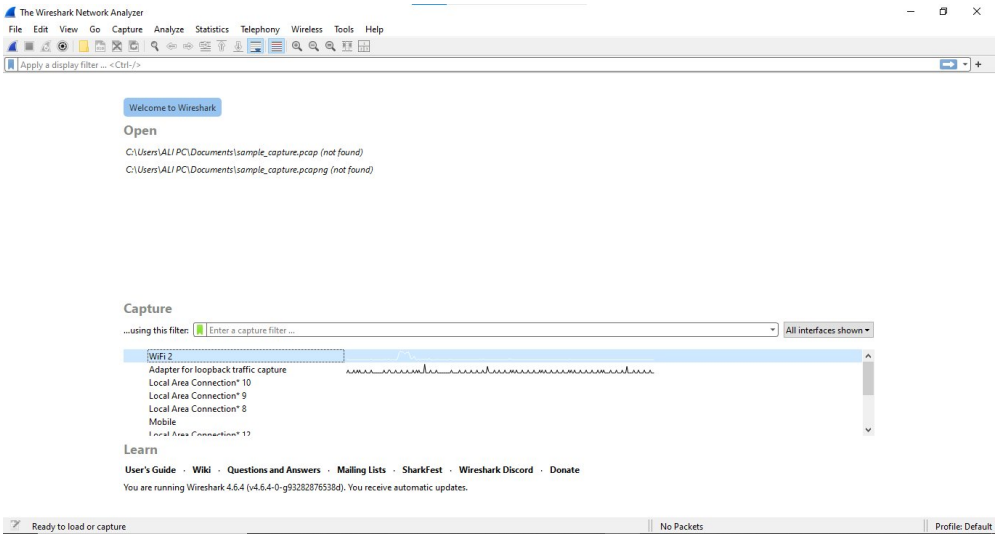

Figure 2 – Wireshark 4.6.4 home screen showing available capture interfaces

2. How to Approach a Packet Capture

Step 1: Define Your Goal

- Identify what you are investigating (e.g. suspicious DNS beaconing, slow page loads, malware C2).
- Choose the correct capture interface — Wi-Fi adapter for wireless traffic, loopback for local processes.

Step 2: Apply a Capture Filter (optional)

- Capture filters are set *before* the capture starts and drop unwanted packets at the OS level.
- Example: `port 53` captures only DNS traffic — reduces file size significantly.

Step 3: Start & Stop the Capture

- Click the shark-fin icon (■) or press **Ctrl+E** to start.
- Reproduce the event or wait for the anomaly, then stop with **Ctrl+E** again.

Step 4: Save the Capture File

- Go to **File** → **Save As**, choose a meaningful name such as `capture.pcap`.
- Use the default **Wireshark/tcpdump .pcap** format for maximum compatibility.

Step 5: Apply Display Filters

- Display filters are applied *after* capture and are non-destructive.
- Example: type `dns` in the filter bar to show only DNS packets.
- Combine filters: `dns and ip.addr==10.251.223.189`

Step 6: Inspect Packets

- Click any packet to expand its layers in the detail pane.
- The hex/ASCII dump on the right shows raw bytes for manual verification.

Step 7: Export & Share

- Use **File** → **Export Specified Packets** to create a filtered subset for sharing.
- The .pcap file can be passed to tools like PyShark, tshark, or Zeek for automated analysis.

Saving the Capture File

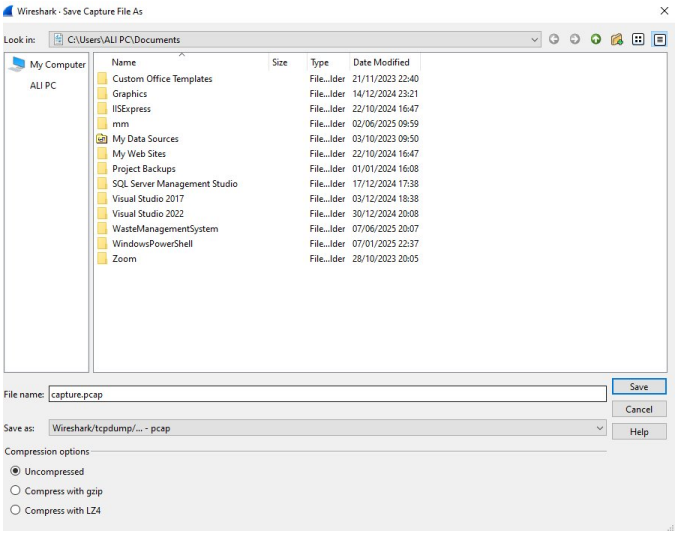

Figure 3 – Saving the capture as 'capture.pcap' in Wireshark

Live Captured Packets

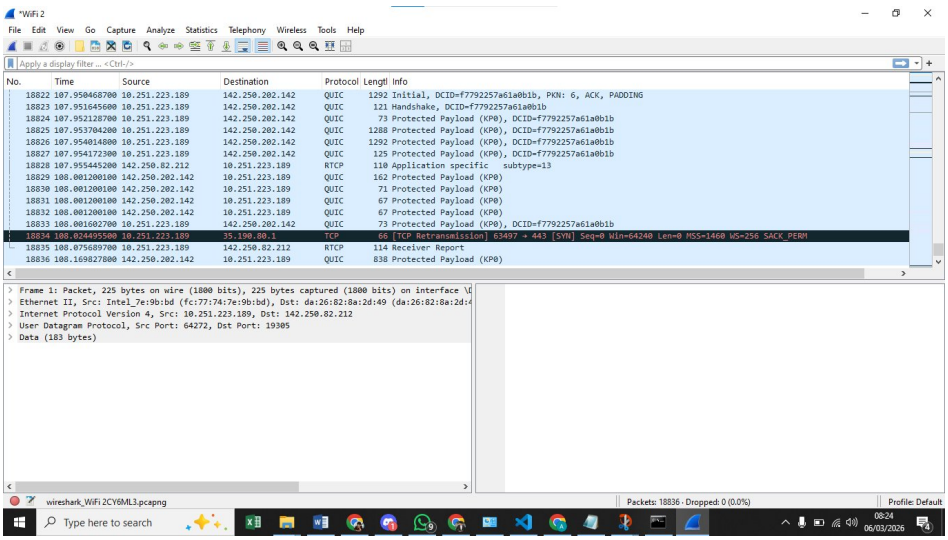

Figure 4 – Live packet list showing QUIC, RTCP and TCP traffic during a WiFi capture

3. Protocol Deep Dive: DNS

What is DNS?

The **Domain Name System (DNS)** is the internet's phonebook. It translates human-readable domain names (e.g. `chatgpt.com`) into IP addresses that routers can forward. DNS operates over **UDP port 53** (and TCP port 53 for large responses or zone transfers).

Why DNS Matters to SOC Analysts

- Malware uses DNS to find its Command & Control (C2) servers — often the only network indicator not blocked.
- DNS tunnelling can exfiltrate data byte-by-byte inside seemingly legitimate queries.
- Unusual query volumes, long sub-domains, or queries to uncommon TLDs are red flags.
- Domain Generation Algorithms (DGAs) produce random-looking names; DNS logs reveal them.

DNS Packet Structure

Transaction ID	16 bits	Matches query to response
Flags	16 bits	QR, Opcode, AA, TC, RD, RA, RCODE
Questions	variable	Domain name + QTYPE (A, AAAA, CNAME, MX, TXT…)
Answer RRs	variable	Resource records in responses
Authority RRs	variable	Nameserver records
Additional RRs	variable	OPT records (EDNS0), glue records

Filtering DNS in Wireshark

Type `dns` in the Wireshark display filter bar and press Enter. Wireshark will immediately hide all non-DNS packets, leaving only DNS queries and responses visible.

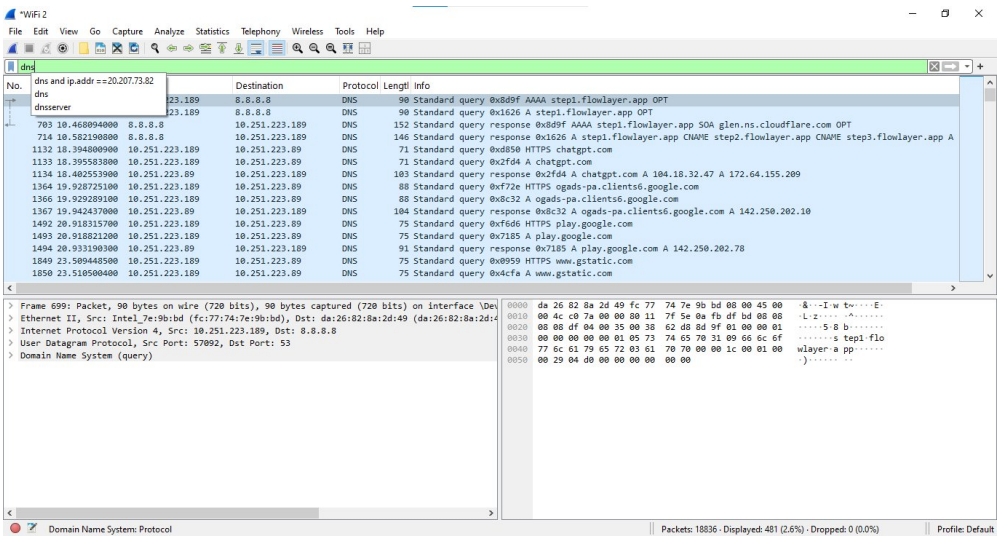

Figure 5 – Display filter 'dns' applied — 481 of 18,836 packets shown

Inspecting a DNS Packet

Expanding the **Domain Name System** layer in the packet detail pane reveals all fields. Below, packet 1493 shows a standard A-record query for `play.google.com`.

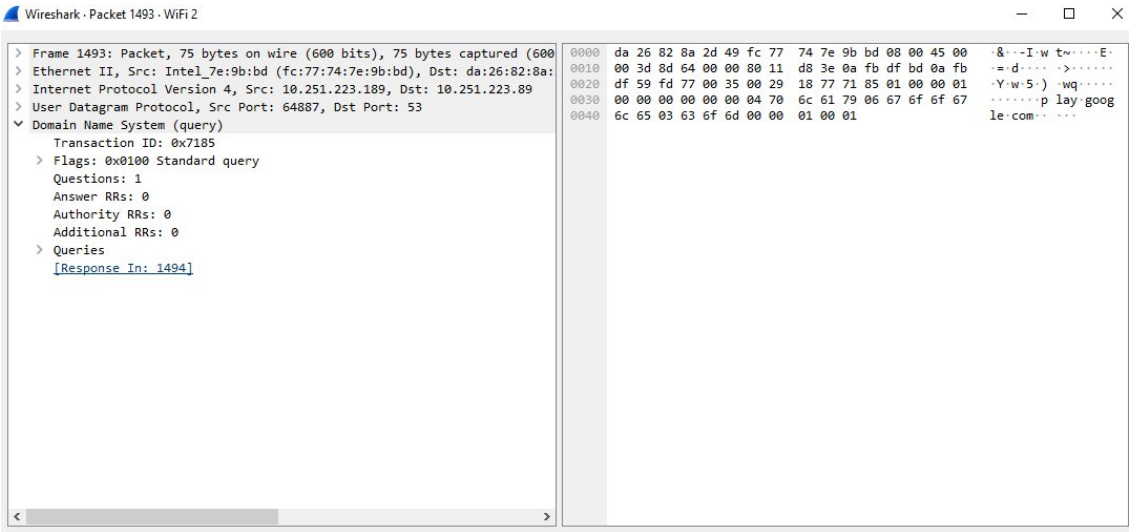

Figure 6 – DNS query packet detail: Transaction ID 0x7185, Flags, and Queries section

4. Python Script: DNS Filter with PyShark

Prerequisites

- **Python 3.x** — standard installation (Windows/Linux/macOS)

  Verify with: `python --version`
- **TShark 4.6.4** — command-line component of Wireshark

  Verify with: `tshark -v`
- **PyShark** — Python wrapper around TShark

  Install with: `pip install pyshark`

Installing TShark & PyShark

```
C:\Users\ALI PC>tshark -v
TShark (Wireshark) 4.6.4 (v4.6.4-0-g93282876538d).

Copyright 1998-2026 Gerald Combs <gerald@wireshark.org> and contributors.
Licensed under the terms of the GNU General Public License (version 2 or later).
This is free software; see the file named COPYING in the distribution. There is
NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Compile-time info:
 Bit width: 64-bit
  Compiler: Microsoft Visual Studio 2022 (VC++ 14.44, build 35223)
      GLib: 2.86.3
 With:
  +brotli                      +MaxMind
  +Gcrypt 1.11.2-unknown       +nghttp2 1.65.0
  +GnuTLS 3.8.12 and PKCS#11   +nghttp3 1.8.0
  +Kerberos (MIT)              +PCRE2 10.47 2025-10-21
  +libpcap                     +Snappy 1.1.9
  +libsmi 0.5.0                +xxhash 0.8.3
```

Figure 7 – TShark 4.6.4 verified from the command line

```
Command Prompt
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ALI PC>pip install pyshark
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pyshark in c:\users\ali pc\appdata\roaming\python\python312\site-packages (0.6)
Requirement already satisfied: lxml in c:\users\ali pc\appdata\roaming\python\python312\site-packages (from pyshark) (6.
0.2)
Requirement already satisfied: termcolor in c:\users\ali pc\appdata\roaming\python\python312\site-packages (from pyshark
) (3.3.0)
Requirement already satisfied: packaging in c:\users\ali pc\appdata\roaming\python\python312\site-packages (from pyshark
) (26.0)
Requirement already satisfied: appdirs in c:\users\ali pc\appdata\roaming\python\python312\site-packages (from pyshark)
(1.4.4)

[notice] A new release of pip is available: 24.0 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\ALI PC>
```

Figure 8 – PyShark installed successfully via pip

The dns_filter.py Script

The script below opens a .pcap file, filters for DNS traffic, and prints a formatted table showing the source IP and queried domain name for every DNS packet found.

```
import pyshark PCAP_FILE = "capture.pcap" def analyze_dns(pcap_file): cap =
pyshark.FileCapture( pcap_file, display_filter='dns' ) print("=" * 55) print(" DNS TRAFFIC
ANALYSIS") print(f"File: {pcap_file}") print("=" * 55) print(f"{'#':<5} {'SOURCE IP':<20}
{'QUERIED DOMAIN'}") print("-" * 55) count = 0 for pkt in cap: try: src_ip = pkt.ip.src #
Extract the first query name domain = pkt.dns.qry_name count += 1 print(f"{count:<5}
{src_ip:<20} {domain}") except AttributeError: continue cap.close() print("-" * 55)
print(f"Total packets analysed : {count}") print(f"DNS queries found : {count}") print("="
* 55) if __name__ == "__main__": analyze_dns(PCAP_FILE)
```

How the Script Works

- **pyshark.FileCapture()** — Opens the .pcap file. The `display_filter='dns'` parameter tells TShark to pass only DNS packets to Python, mirroring what you would type in Wireshark's filter bar.
- **pkt.ip.src** — Accesses the source IP address from the IP layer of each packet.
- **pkt.dns.qry_name** — Extracts the queried domain name from the DNS layer. Raises `AttributeError` for response-only packets (no query), which we catch and skip.
- **cap.close()** — Releases the file handle and TShark subprocess cleanly.

Script Output

Running `python dns_filter.py` from the project directory produces a clean tabular report. The first screen shows entries 1–16:

```
C:\Users\ALI PC\Desktop\wireshark-task>python dns_filter.py
=======================================================
                 DNS TRAFFIC ANALYSIS
File: capture.pcap
=======================================================
#     SOURCE IP            QUERIED DOMAIN
-------------------------------------------------------
1     10.251.223.189       step1.flowlayer.app
2     10.251.223.189       step1.flowlayer.app
3     8.8.8.8              step1.flowlayer.app
4     8.8.8.8              step1.flowlayer.app
5     10.251.223.189       chatgpt.com
6     10.251.223.189       chatgpt.com
7     10.251.223.89        chatgpt.com
8     10.251.223.189       ogads-pa.clients6.google.com
9     10.251.223.189       ogads-pa.clients6.google.com
10    10.251.223.89        ogads-pa.clients6.google.com
11    10.251.223.189       play.google.com
12    10.251.223.189       play.google.com
13    10.251.223.89        play.google.com
14    10.251.223.189       www.gstatic.com
15    10.251.223.189       www.gstatic.com
16    10.251.223.89        www.gstatic.com
17    10.251.223.89        chatgpt.com
```

Figure 9 – dns_filter.py output (rows 1–16): source IPs and queried domains

The script processed all 481 DNS packets from the capture file:

```
Command Prompt
479   10.251.223.189       a.nel.cloudflare.com
480   10.251.223.189       a.nel.cloudflare.com
481   10.251.223.89        a.nel.cloudflare.com
-------------------------------------------------------
Total packets analyzed : 481
DNS queries found      : 481
=======================================================

C:\Users\ALI PC\Desktop\wireshark-task>_
```

Figure 10 – Final summary: 481 packets analysed, 481 DNS queries found

■ **Observations from the Capture:** The analysis revealed DNS queries to **chatgpt.com**, **play.google.com**, **ogads-pa.clients6.google.com**, **www.gstatic.com**, **step1.flowlayer.app**, and **a.nel.cloudflare.com**. Multiple source IPs (`10.251.223.189` and `10.251.223.89`) generated the queries, all resolved through Google's public DNS server `8.8.8.8`.

5. Quick Reference

Common Wireshark Display Filters

`dns`	All DNS traffic
`tcp`	All TCP traffic
`udp`	All UDP traffic
`http`	All HTTP traffic
`tls`	All TLS/SSL traffic
`icmp`	All ICMP (ping) traffic
`ip.addr == 10.251.223.189`	All traffic to/from a specific IP
`dns and ip.src == 10.x.x.x`	DNS queries from a specific host
`tcp.port == 443`	HTTPS traffic
`frame contains "password"`	Frames containing the string 'password'

DNS Query Types

Type	Purpose
A	IPv4 address lookup
AAAA	IPv6 address lookup
CNAME	Canonical name / alias
MX	Mail exchange record
TXT	Text records (SPF, DKIM, DMARC)
PTR	Reverse DNS lookup
SOA	Start of Authority
NS	Nameserver record
HTTPS	HTTPS service record (modern browsers)

Keyboard Shortcuts

`Ctrl+E`	Start / Stop capture
`Ctrl+S`	Save capture file
`Ctrl+F`	Find packet
`Ctrl+G`	Go to packet number
`Ctrl+→ / Ctrl+←`	Next / Previous packet in selection
`Spacebar`	Scroll down one page
`Alt+→`	Follow TCP/UDP stream

■ **GitHub Repository:** All scripts, the capture.pcap file, and further lab write-ups are available at https://github.com/aliwaqar04/CyberConnect-Fellowship-SOC-labs